

CRYME — Sprechernotizen

Masterprojekt Informatik · Erick Zeiler · Ziel: ca. 20 Minuten

1. Zeitplan (Übersicht)

Nr.	Folie	Dauer	Kumuliert
1	Titel — CRYME	0:45	0:45
2	Motivation und Forschungsfrage	2:00	2:45
3	Post-Quanten-Standards (NIST)	1:30	4:15
4	Was ist CRYME?	1:30	5:45
5	Architektur und Domänenmodell	1:30	7:15
6	Tech-Stack & Implementierung	2:00	9:15
7	Oracle-Logik im Detail	2:00	11:15
8	Demo-Setup	1:00	12:15
9	Demo Schritt 1 — FAIL	2:30	14:45
10	Demo Schritt 2 — SUCCESS	2:00	16:45
11	Migrationsbaum und weitere Schritte	1:30	18:15
12	Ergebnisse	0:45	19:00
13	Grenzen, Fazit und Ausblick	0:45	19:45
14	Vielen Dank / Fragen	0:15	20:00

Tipp: Demo-Folien 8–10 = Kern. Kürzen bei Zeitdruck: F5 (Architektur) oder F12 (Ergebnisse). Live-Demo → F8 kürzer, Zeit in F9–10.

2. Sprechernotizen pro Folie

Folie 1 CRYME — Titelfolie

1 Min.

- Begrüßung · Name · Masterprojekt
- **CRYME** = Cryptographic Migration Engineering
- Simulationsbasiertes Oracle für PQC-Migration
- Pipeline: **YAML Twin** → **Oracle** → **Deploy** → **Verify** (roter Faden)
- Betreuer · HSR · Ablauf: Problem → Lösung → Demo

Folie 2 Motivation und Forschungsfrage

2:00 Min.

- **Warum PQC?** Quantencomputer → RSA/ECC bedroht (Shor)
- Migration ≠ Algorithmus-Tausch → **Systemproblem**
- Abhängigkeiten:
 - Server ↔ Client (KEX)
 - Zertifikat ↔ Schlüssel (Trust Chain)
 - TLS-Version ↔ KEX-Algorithmus
- Naive Einzel-Migration → Systemausfall möglich
- **Forschungsfrage langsam vorlesen**
- → Überleitung: „Welche Algorithmen kommen zum Einsatz?“,

Folie 3 Post-Quanten-Standards (NIST)

1:30 Min.

- NIST 2024 — PQC wird Standard, nicht Option
- **ML-KEM** (FIPS 203) — Key Encapsulation · ersetzt ECDHE
- **ML-DSA** (FIPS 204) — Signaturen · ersetzt RSA/ECDSA
- **Hybrid-KEX**: X25519_MLKEM768 — Classic + PQC parallel
- Kein Big-Bang — schrittweise Phasen (KEX → Cert → TLS)
- Migrationspfad-Tabelle durchgehen
- Jeder Schritt = Live-System ändert sich → Risiko
- → „CRYME orchestriert genau diesen Pfad sicher“,

Folie 4 Was ist CRYME?

1:30 Min.

- **Orchestrator** — plant · validiert · deployt
- Ersetzt **keinen** TLS-Stack (OpenSSL/nginx)
- Pipeline: YAML → Graph → Oracle → Ansible → Live
- CLI:
 - **init** — System starten
 - **migrate** — Schritt simulieren
 - **deploy step=N** — validierten Schritt ausführen
 - **show tree** — Migrationshistorie

- 3 Mechanismen:
 - **Dependency Discovery** → implizite Kanten → SCC-Cluster
 - **Temporal Barriers** → keine unsicheren Zwischenzustände
 - **Event Sourcing** → Replay zu Schritt N
- Kernprinzip: **Prüfen vor Ausführen**

Folie 5 Architektur und Domänenmodell

1:30 Min.

- Komponenten:
 - **cryme CLI** — Orchestrator
 - **Memgraph** — Graph-DB + Historie
 - **oracle.js** — SCC · implizite Kanten · Temporal Rules
 - **nginx :8443** — live migrierbarer HTTPS-Dienst
- Pipeline: YAML → migrate → Oracle → deploy → Ansible → nginx
- YAML-Modell:
 - Components (Webserver, Browser)
 - CryptoAssets (Zertifikat, KEX)
 - SecurityControls (TLS-Version)
 - PQC Variants (ML-KEM, ML-DSA)
- PoC: `webserver_pqc_twin.yaml`
- Oracle-Regeln kurz nennen → Details auf F7

Folie 6 Tech-Stack & Implementierung

2:00 Min.

- **Anwendung:**
 - Node.js — CLI + Oracle (1300 LOC oracle.js)
 - neo4j-driver — Bolt zu Memgraph
 - js-yaml — Twin-Parsing
- **Infrastruktur:**
 - Docker Compose — 4 Container + Memgraph Lab
 - nginx 1.27 — TLS-Termination
 - curl-client — Browser-Ersatz
- **Deploy:**
 - Ansible Rolle `cryme_tls` — Profil aus Graph-Zustand
 - Generierte Playbooks pro Schritt
- **Verify:**
 - OpenSSL + curl — unabhängig von cryme
 - `verify_tls.sh` — Cipher-Suite + `/api/status`
- Warum Memgraph? — leichtes Docker-Setup, Cypher, PoC-tauglich
- Konzept DB-agnostisch (Neo4j-kompatibel via Bolt)

Folie 7 Oracle-Logik im Detail

2:00 Min.

- **3 Kantentypen:**
 - Explizit — `depends_on` in YAML
 - Implizit — `SecurityControl` → `CryptoAsset`
 - Discovered — beim FAIL neu gelernt (Server ↔ Browser)
- **SCC (Tarjan):**
 - Starke Zusammenhangskomponenten
 - Cluster = Co-Migrations-Pflicht
 - Ein Befehl → Oracle erweitert auf ganzen Cluster
- **Temporal Barriers:**
 - `not_before`-Constraints
 - Beispiel: TLS 1.3 only erst nach KEX-Migration
 - Verletzung → ABORTED (Schritt 3 in Demo)
- **Deny-by-default** — unsicher = FAILED, nie silent pass
- Event Sourcing — jeder Versuch dokumentiert
- → „Jetzt sehen wir das live,,

Folie 8 Demo-Setup

1:00 Min.

- Server **ilmare** — 4 Container:
 - Memgraph · cryme CLI · nginx :8443 · curl-Client
- Migrationsziel: Classic → Hybrid-KEX → ML-DSA → TLS 1.3 only
- Verify: curl + openssl (unabhängig von cryme API)
- **Baseline Schritt 0:**
 - Zertifikat: RSA-2048
 - KEX: ECDHE (Server + Client)
 - TLS: 1.2 / 1.3
 - Profil: `classic-rsa-ecdhe`
- Zwei Sichten: `show state step=N` ↔ TLS-Handshake :8443
- → „Was passiert bei falschem Schritt?,,

Folie 9 Demo: Schritt 1 — Versteckte Abhängigkeit (FAIL) 2:30 Min.

- Befehl: nur Server-KEX migrieren
 - `Webserver.KeyExchange_ECDHE` → `X25519_MLKEM768`
- Ergebnis: **FAILED** — **Policy Denied**
- Server = PQC · Client = noch Classic (ECDHE)
- **Warum?**
 - Implizite Kante Server ↔ Browser
 - KEX-Mismatch → TLS-Handshake bricht
- Oracle blockiert **präventiv** (deny-by-default)

- **Kernaussage — langsam, Pause:**

- Versteckte Abhängigkeit erkannt
- Bevor Live-System betroffen
- Thesis-Nachweis 1 ✓
- → „Was stattdessen?“,

Folie 10 Demo: Schritt 2 — Co-Migration (SUCCESS)

2:00 Min.

- Gleicher Befehl — anderes Ergebnis
- Oracle findet SCC-Cluster → migriert **beide** Endpunkte
- Ergebnis: **SUCCESS — Co-Migrated Cluster**
 - Server + Browser → X25519_MLKEM768
 - Profil: hybrid-kex-classic-cert
- Intern: SCC-Expansion über Kommunikationskante
- `deploy step=2` → nginx live
- Verify: API + TLS-Handshake (Hybrid-KEX)
- Kernstory: FAIL → SUCCESS → Live-Deploy ✓

Folie 11 Migrationsbaum und weitere Schritte

1:30 Min.

- `cryme show tree`:
 - [0] INIT
 - [1] FAILED (Einzelversuch)
 - [2] SUCCESS (Co-Migration)
 - [3] ABORTED (Zertifikat — Vorbedingung fehlt)
 - [4] SUCCESS (Cert → ML-DSA-44)
 - [5] SUCCESS (TLS 1.3 only)
- Jeder Schritt: migrate → deploy → verify
- **Event Sourcing:**
 - Alle Versuche dokumentiert (auch FAIL)
 - Replay zu Schritt N
 - Git-Analogie: Commit · Log · HEAD
- Audit-Trail für kryptografische Migrationen

Folie 12 Ergebnisse

0:45 Min.

- Thesis-Claims ✓:
 - Versteckte Abhängigkeiten erkannt (Schritt 1)
 - SCC-Co-Migration (Schritt 2)
 - Live-Deploy — API + TLS ändern sich
 - Unabhängige Verify (curl/openssl)

- TLS-Profil:
 - classic-rsa-ecdhe → Start
 - hybrid-kex-classic-cert → nach Schritt 2
 - hybrid-kex-mldsa-cert → nach Cert-Migration
 - tls13-only → Ende
- Neue Algorithmen via `migration_variants` in YAML — kein Code-Change
- **Kurz halten — Demo hat schon gezeigt**

Folie 13 Grenzen, Fazit und Ausblick

0:45 Min.

- **PoC-Grenzen:**
 - 1 Szenario (Webserver + Browser)
 - ML-DSA-Zertifikate experimentell
 - Keine Auto-Planung (Operator gibt Schritte vor)
 - Stack: Docker · Memgraph · Ansible
- **Fazit:**
 - Riskante Schritte **vor** Ausführung erkennen + abfangen
 - Formel: Abhängigkeiten → Clusters → sichere Reihenfolge
- **4 Beiträge:**
 - Oracle-Framework
 - Graphbasiertes SCC-Clustering
 - End-to-End: YAML → Deploy → Verify
 - Event Sourcing + Audit-Trail
- **Ausblick:** beliebige Landschaften · Auto-Planung · Crypto-as-Code
- Forschungsfrage: Ja ✓

Folie 14 Vielen Dank / Fragen

0:15 Min.

- Danke · Fragen?
- Optional: Live-Demo wiederholen (:8443)
- Ruhe bewahren — nicht nachreden

3. Notizen zur Präsentationsführung

Zeitmanagement

- 20 Min. · Fragen danach
- Zu langsam → kürzen: F5 (Architektur), F12 (Ergebnisse)
- Zu schnell → mehr Zeit: F6–7 (Tech/Oracle), F9–10 (Demo)
- Rede-Folien: F3, F6, F7 — hier kannst du ausführlicher werden

Tempo

- Langsamer: Forschungsfrage (F2) · Kernaussage (F9)
- CLI nicht vorlesen — paraphrasieren
- Pause nach FAILED (F9)

Live-Demo

- F8: 30s · F9–10: je 3 Min.
- Testlauf: init → curl → migrate FAIL → migrate SUCCESS → deploy step=2
- Fallback: Folien-Screenshots reichen

Typische Fragen

- Memgraph statt Neo4j? → Docker-Setup · Konzept DB-agnostisch
- Skalierung? → Prinzip bewiesen · Auto-Planung fehlt
- Schritt 3 ABORTED? → Temporal Barrier · Vorbedingung fehlt